NATIONAL POLYTECHNIC UNIVERSITY OF ARMENIA

Faculty of Applied Mathematics and Physics

Optimal Allocation of Energy Resources of a Grid Connected Microsystem: Nonlinear Case

Faculty/institute: Informatics and Applied Mathematics (Computer Science)

Group: MT240a

Student: Vany Kardjikian

Supervisor: I. Hovhannisyan

YEREVAN 2026

Table of Contents

Introduction.....4
Chapter 1: Literature Review.....5
1.1. Study of Existing Models..... 5
1.2 Definitions of Key Concepts:..... 5
1.3. Optimization Algorithms Relevant to the Problem:..... 6
1.3.1 Simplex..... 6
1.3.2 Quadratic Programming.....7
1.3.3 Sequential Quadratic Programming:.....7
1.3.4 Branch and bound..... 9
1.3.6 Branch-and-Bound Method for Mixed-Integer Nonlinear Programming (MINLP) 10
1.4 Tools and Software.....11
1.4.1 Python..... 11
1.4.2 Libraries..... 12
Chapter 2: Problem Statement and Mathematical Formulation..... 14
2.1 Problem Statement..... 14
2.2.1 Mathematical Model..... 15
2.2.2 Solution Methodology:..... 17
2.3. Nonlinear Programming Model:..... 18
2.3.1 Mathematical Model:..... 18
2.3.2 Solution Methodology:..... 19
Chapter 3: Software Implementation and Code Structure.....21
3.1 Code Structure..... 21
3.2 Inputs and Outputs..... 24
Chapter 4: Case Study: Small Commercial Building in Yerevan.....26
4.1 System parameters..... 26
4.2 Results.....28
4.3 Discussion..... 32
Conclusion..... 39
References.....40
Chapter 5: Safety Issues of Optimal Allocation of Energy Resources of a Grid-Connected Microsystem.....42
Introduction.....43
5.1 Safety Measures During Installation and Maintenance of PV Systems..... 44

5.2 Safety Faults in Photovoltaic Systems.....46
5.3 Fire Safety and Emergency Response in PV Systems.....49
5.4 String Cables Safety Calculations.....50
Conclusion.....52
References.....53
Chapter 6: The Environmental Impacts of Using Small-Scale Solar Photovoltaic Systems... 54
Introduction.....55
6.1 Environmental Impacts of Solar PV Systems.....56
6.3 Energy Payback Time (EPBT).....57
6.4 Life Cycle Assessment (LCA).....58
Conclusion.....58

Introduction

Today, more and more people are becoming interested in sustainable energy systems for various reasons. Whether it is to save the environment or save money, solar energy remains one of the most sought-after sources.

In recent years, Armenia has been experiencing rapid growth in solar generation. Currently, thousands of commercial and residential buildings in Armenia have rooftop photovoltaic systems installed. Time-of-use tariffs have introduced a significant price difference between day and night, but the export price for surplus solar energy remains low. Some systems even add a lithium-ion battery allowing the owner to store excess solar energy and use it during expensive peak hours or sell it back to the grid.

With such a system in place, the question remains: how to optimally utilize it. In order to achieve the maximum possible daily profit or minimize the cost of electricity, the owner needs to make the correct decision at every single hour of the day: charge, discharge, buy from the grid, or sell to the grid. This decision problem is complex because the battery cannot charge and discharge at the same time. In addition to that, real inverters and batteries have losses that change in a nonlinear way.

The goal of this thesis is to create a mathematical optimization model that finds the best 24-hour schedule for a building. A case study is also included for a commercial building in Yerevan under 2026 tariffs, with calculations on how much money can be saved with this system. Realistic nonlinear losses are taken into account and compared to a simplified linear model.

Calculations are performed using open-source Python tools: PuLP + CBC for the linear case and GEKKO (APOPT) for the nonlinear case.

Chapter 1: Literature Review

1.1. Study of Existing Models

Optimal daily energy management of PV-battery systems under time-of-use tariffs has been a widely studied problem. Most works use mixed-integer linear programming (MILP) assuming constant efficiency. They achieve a reduced cost compared to grid-only operation.

However, real inverter and battery losses are quadratic in power and ignoring them overestimates savings. Such problems are formulated as convex mixed-integer nonlinear programs (MINLP).

This thesis develops and compares linear and nonlinear models, solves them using open-source tools, and calculates the impact of quadratic losses for a commercial building in Yerevan under different export compensation scenarios.

1.2 Definitions of Key Concepts:

Mathematical Programming Problem:

A mathematical programming problem is finding values of the decision variables that minimize (or maximize) an objective function, subject to a set of constraints [12].

$$\begin{cases} f(\vec{x}) \to \min(\max), \\ g_i(\vec{x}) = 0, \ i = \overline{1, m}, \end{cases} \tag{1}$$

Linear Programming Problem:

A linear programming problem is a mathematical optimization problem in which the objective function and all constraints are linear functions of the decision variables [12].

Nonlinear Programming Problem:

A nonlinear programming problem is a mathematical optimization problem in which at least one of the objective function or constraints is nonlinear [12].

Mixed-Integer Programming (MIP) Problem:

A mixed-integer programming (MIP) problem is a mathematical optimization problem in which some of the decision variables are required to take integer values, while the remaining variables may be continuous [12].

Gradient of a function:

For a given function f, the first order gradient is the following vector [12]:

$$\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}(\mathbf{x}), \frac{\partial f}{\partial x_2}(\mathbf{x}), \cdots, \frac{\partial f}{\partial x_n}(\mathbf{x}) \right)$$

Lagrangian Function:

Given the problem (1), the lagrangian function is defined as [12]:

$$L(\vec{x}, \vec{\lambda}) = f(\vec{x}) + \lambda_1 g_i(\vec{x}) + \lambda_2 g_2(\vec{x}) + \cdots + \lambda_m g_m(\vec{x})$$

1.3. Optimization Algorithms Relevant to the Problem:

1.3.1 Simplex

The Simplex algorithm, introduced by George Dantzig, is a method for solving linear programming (LP) problems. The algorithm finds the optimal value of a linear objective function by searching the vertices of the feasible region, which is a convex polyhedron defined by linear constraints.

The set of feasible solutions in an LP problem is convex, meaning any optimal solution lies at an extreme point (vertex) of the feasible polyhedron.

Each extreme point of the feasible region corresponds to a basic solution, which is defined by selecting a set of basic variables while keeping the remaining non-basic variables at zero.

<u>Algorithm</u>

The Simplex method operates through iterative pivoting between dictionaries. An entering variable is selected from the non-basic variables to increase (or decrease, in minimization) the objective function. A leaving variable is selected from the basic variables to maintain feasibility (non-negativity).

The selected entering and leaving variables are exchanged through row operations to update the dictionary progressing toward the optimal solution. Iteration continues until no further improvement is possible, which occurs when all coefficients in the objective function row of the tableau are non-negative (for maximization problems) [6].

1.3.2 Quadratic Programming

Quadratic programming (QP) is a programming problem where the objective function is quadratic and the constraints are linear [12].

$$\begin{cases} f(\vec{x}) = \frac{1}{2}\vec{x}Q\vec{x} + \vec{r}\vec{x} \to min, \\ A\vec{x} \leq \vec{b}, \\ \vec{x} \geq \vec{0}. \end{cases}$$

If Q is a symmetric positive semidefinite matrix, the problem can be solved using KKT (Karush–Kuhn–Tucker) conditions [12].

1.3.3 Sequential Quadratic Programming:

Sequential quadratic programming (SQP) is a class of algorithms that can solve nonlinear optimization problems (NLP) in the real world. It can handle any degree of nonlinearity including nonlinear constraints [15].

$$\begin{aligned} & f(\mathbf{x}) \longrightarrow \boldsymbol{min} \\ & g_i(\mathbf{x}) = 0 \quad i = 1,\ldots,m \end{aligned}$$

In order to make the problem easier to solve, a QP approximation is a method used where the nonlinear optimization problem is approximated by a simpler quadratic problem at each iteration.

The quadratic approximation at the point x^k is:

$$\begin{aligned} & f_a = f^k + \left(\nabla f^k\right)^{\mathrm{T}} \Delta\mathbf{x} + \frac{1}{2}\left(\Delta\mathbf{x}\right)^{\mathrm{T}} \nabla_x^2 \mathrm{L}^k \Delta\mathbf{x} \quad \longrightarrow min \\ & g_i^k + \left(\nabla g_i^k\right)^T \Delta\mathbf{x} = 0 \quad i = 1,\ldots,m \end{aligned}$$

Now to solve the new problem, we will solve the KKT equations for this problem.

$$\nabla f + \sum_{i=1}^{m} \lambda_i \nabla g_i = 0$$

$$g_{i,a} = 0 \quad i = 1,2,\ldots,m$$

SQP Algorithm [15]:

a. Initialization

The algorithm begins with an initial estimate x^0 of the decision variables, which serves as the starting point for the iterative optimization process.

b. Quadratic Programming (QP) Approximation

A quadratic programming approximation of the original nonlinear optimization problem is constructed. In the first iteration, the Hessian matrix of the Lagrangian is initialized as the identity matrix.

c. Solution of the QP Subproblem

The resulting QP subproblem is solved to obtain a search direction.

d. Line Search

A line search is performed along the computed search direction to improve the value of the objective function. The step is defined as:

$$\mathbf{x}^{new} = \mathbf{x}^{old} + \Delta\mathbf{x}$$

Where Δx is the initial step.

A penalty function is evaluated:

$$P = f + \sum_{i=1}^{vio} \lambda_i |g_i|$$

where the summation is taken over the violated constraints.

e. Evaluation and Hessian Update

The gradient of the Lagrangian is evaluated at the new iterate. The differences in the decision variables and in the Lagrangian gradients are computed. The Hessian approximation of the Lagrangian is then updated using the BFGS (Broyden-Fletcher-Goldfarb-Shanno) method.

f. Iterative Process

Steps 2 through 5 are repeated until the step Δx becomes sufficiently small. As Δx approaches 0, the Karush-Kuhn-Tucker (KKT) optimality conditions are satisfied, the optimal solution has been found.

1.3.4 Branch and bound

Branch and Bound is a tree-based method for solving discrete and combinatorial optimization problems, including mixed-integer linear programming (MILP) and mixed-integer nonlinear programming (MINLP). The method constructs a state space tree, where each node represents a partial solution that is obtained by fixing a subset of decision variables.

The algorithm has three fundamental operations [8]:

- **Generation of the State Space Tree (Branching):** The original problem is divided into subproblems by fixing integer decision variables. Each branching step creates child nodes corresponding to alternative decisions (e.g., $x_i = 0$ or $x_i = 1$ in binary problems). All children of expanded nodes are generated forming a structured search tree.
- **Evaluation of Candidate Nodes (Bounding):** Each node is evaluated by solving a relaxed version of the corresponding subproblem. The relaxation provides a bound on the best achievable objective value within that branch. In minimization problems, the relaxation yields a lower bound and in maximization problems, it yields an upper bound.
- **Pruning (Node Elimination):** A node becomes inactive (dead) if its relaxation is infeasible or its bound is worse than the best feasible integer solution. Pruning reduces computational effort by eliminating subproblems that cannot improve the objective value.

There are different search strategies within the search tree based on the order in which nodes are explored [8].

- **FIFO (First-In-First-Out):** Uses a queue structure and performs breadth-first search.
- **LIFO (Last-In-First-Out):** Uses a stack structure and performs depth-first search.
- **Least-Cost (Best-Bound) Strategy:** Selects the node with the most promising bound.

1.3.5 Branch-and-Cut Method

The branch-and-cut (B&C) method is the most dominant approach in modern MILP solvers, It is an algorithmic framework for solving mixed-integer linear programs (MILPs). B&C is an extension of the classical branch-and-bound (B&B) algorithm. It dynamically incorporates cutting planes to strengthen the relaxations at nodes of the search tree, this reduces the number of nodes explored compared to pure branch-and-bound [11].

Algorithm [11]:

Branch-and-cut operates on a search tree where each node represents a subproblem obtained by adding integrality constraints and/or additional bounding constraints to the original problem.

1. At each node, an LP relaxation is yielded.
2. The LP is solved to obtain a lower bound on the optimal value in that subproblem and a solution.
3. If the LP solution is integer-feasible and satisfies all original constraints, it provides an upper bound for the overall problem.
4. If the solution is fractional, the algorithm identifies violated valid inequalities, linear constraints satisfied by all integer-feasible points but violated by the current fractional solution.
5. Cuts are added to the current LP relaxation, and the strengthened LP is re-solved.
6. The cutting process repeats until no violated cuts are found or the solution becomes integer.

Types of Cutting Planes [11]:

Different types of cutting-planes are used, based on the problem: Gomory cuts (derived from the simplex tableau, applicable in general MILPs [12]), Cover cuts, Clique cuts and others.

It is also important to distinguish between:

- Global cuts: Valid across the entire tree.
- Local cuts: Valid only within a subtree.
- Lazy cuts: Generated only when needed.

1.3.5 Branch-and-Bound Method for Mixed-Integer Nonlinear Programming (MINLP)

The branch-and-bound method was originally proposed by Land and Doig (1960) for discrete programming problems. It has been extended to solve mixed-integer nonlinear programs (MINLPs) [5].

The general MINLP solved by branch-and-bound takes the following form where f and the components of g and h are continuously differentiable, but potentially nonconvex [5].

$$
\begin{aligned}
\min_{x,y} \quad & f(x,y) \\
\text{subject to} \quad & g(x,y) \leq 0, \\
& h(x,y) = 0, \\
& x \quad \in [x^L, x^U] \subseteq \mathbb{R}^n, \\
& y \quad \in \{0,1\}^m \quad (\text{or } y \in \mathbb{Z}^m \text{ in more general cases}).
\end{aligned}
$$

Branch-and-bound for MINLP relies on two fundamental operations: relaxation and search. The algorithm builds a search tree similar to the one described in 1.3.3.

<u>Algorithm [5]:</u>

1. Set upper bound $U=+\infty$ and lower bound $L=-\infty$. Initialize binary variables. Solve the NLP relaxation with fixed y. if feasible, obtain an initial feasible solution and upper bound; otherwise, update y and repeat until feasibility or prove infeasibility of the MINLP.

2. For a selected node with fixed or partial integer variables, solve the corresponding NLP relaxation to obtain a lower bound.

3. Select an unsolved node from the list. Solve the NLP subproblem. If feasible and integer solution found: update incumbent U and best solution if improved. If fractional, generate Gomory mixed-integer cuts (if violated) to tighten the relaxation, or branch on a fractional integer variable. Add child nodes to the list and continue.

4. The algorithm terminates when no unsolved nodes remain. The best incumbent solution is the global optimum for convex MINLPs.

1.4 Tools and Software

1.4.1 Python

Python is a programming language created by Guido van Rossum in 1991. It works on different platforms and has a simple syntax similar to the English language. Python is used in many areas, such as Web and Internet Development, Scientific and Numeric, Education, Desktop GUIs and Software Development [16].

1.4.2 Libraries

Libraries are collections of modules containing functions to perform common tasks. Python libraries make coding more efficient by providing developers with existing methods that are ready to use.

OS: os is a built-in module that can be used to create files and directories.

Pandas: Pandas is a Python library used to analyze data. It provides fast and flexible tools to work with data. It can be used with other libraries such as Matplotlib and NumPy [9].

Matplotlib: Matplotlib is a library for creating static, animated, and interactive visualizations in Python [13].

PuLP: PuLP is a linear and mixed integer programming modeler. It can be used to create MILP optimization problems and solve them with the latest solvers. PuLP can generate MPS or LP files and call solvers such as GLPK, COIN-OR CLP/CBC, CPLEX, GUROBI, MOSEK, XPRESS, CHOCO, MIPCL, HiGHS, SCIP/FSCIP [4].

PuLP uses the COIN-OR Branch and Cut (CBC) solver as its default solver for mixed-integer programming problems. CBC is an open-source mixed-integer program (MIP) solver written in C++, developed as part of the COIN-OR project and developed historically by John Forrest [3].

The core algorithm used by CBC is the branch-cut-and-bound algorithm described in 1.3.4.

GEKKO: GEKKO is a Python library that can be used for machine learning and optimization of mixed-integer and differential algebraic equations. It is integrated with solvers capable of handling linear programming (LP), quadratic programming (QP), nonlinear programming (NLP), mixed-integer linear programming (MILP), and mixed-integer nonlinear programming (MINLP).

GEKKO provides a high-level framework for creating mathematical models. Model components are defined as constants, parameters. Objective functions are specified to be minimized or maximized.

GEKKO offers nine operational modes, grouped into steady-state and dynamic categories. Steady-state modes (simulation, parameter update, and real-time optimization) assume zero derivatives. Dynamic modes (simulation, estimation, and control) describe time-dependent

system behavior using differential equations. Each dynamic mode can be solved using either a simultaneous or a sequential numerical approach.

The computational engine behind GEKKO is APMonitor, which compiles models into byte-code and performs structural analysis to reduce problem size based on sparsity patterns.

APMonitor supplies nonlinear programming solvers (APOPT, IPOPT, SNOPT, MINOS, and BPOPT) with the necessary model information. After the solution, results are exported in a json format and returned back to GEKKO [10].

Chapter 2: Problem Statement and Mathematical Formulation

2.1 Problem Statement

Modern commercial and residential buildings increasingly combine rooftop photovoltaic (PV) systems, battery energy storage, and grid connection under time-of-use tariffs regulations. These buildings face the challenge of finding the optimal schedule of energy flows on an hourly basis to minimize electricity costs (or maximize financial benefit).

The problem is to determine, for each hour of the day, the most economically beneficial operating strategy given varying electricity demand of the building, hourly solar generation from the rooftop PV system, time-of-use grid purchase prices and export prices, and a lithium-ion battery with limited capacity, charge/discharge power, and efficiency

The system must decide how much electricity to purchase from or sell to the grid as well as battery charge and discharge rates while respecting mutual exclusivity of simultaneous battery charging/discharging, state of charge dynamics and power limits

The objective is to maximize daily electricity cost savings compared to a baseline without local generation or storage.

The problem is solved in two variants:

1. Mixed-integer linear program (MILP) using PuLP/CBC
2. Convex mixed-integer nonlinear program (MINLP) using GEKKO with Outer Approximation (APOPT).

The studied system consists of four main components connected through a single hybrid inverter:

- Rooftop photovoltaic panels (DC output)
- Lithium-ion battery storage (bidirectional DC)
- Building electrical load (AC)
- Utility grid connection (bidirectional AC)

All energy flows are managed by the hybrid inverter, which enables simultaneous solar generation, battery charging or discharging, and grid import or export while respecting the mutual-exclusivity constraint.

The linear model assumes constant efficiency while the nonlinear model introduces a quadratic loss term. Although the nonlinear formulation better reflects physical reality, the linear case serves as an important benchmark and is significantly easier to solve and analyse.

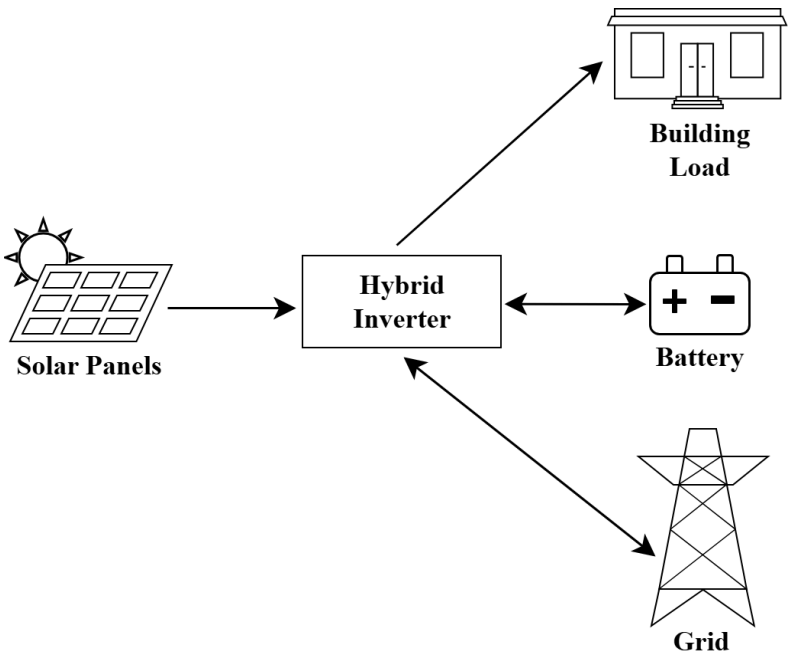

Figure 2.1: Schematic diagram of a grid-connected photovoltaic system with battery energy storage and hybrid inverter

Therefore, to fully understand the more complex nonlinear case and to clearly calculate the impact of realistic losses, we first study a linear model before extending it to a nonlinear one.

2.2 Linear Programming Model

2.2.1 Mathematical Model

Indices

$t = 0, 2, 3, \dots, T$	Hours in the optimization period

Parameters

C_t^{buy}	Electricity purchase price at hour t (AMD/kWh)
C_t^{sell}	Electricity selling price at hour t (AMD/kWh)
E_t^{solar}	Solar generation at hour t (kWh)

E_t^{demand}	Load demand at hour t (kWh)
E^{cap}	Battery capacity (kWh)
$P_{charge}^{max}, P_{discharge}^{max}$	Max battery charge/discharge power (kW)
$P_{buy}^{max}, P_{sell}^{max}$	Max grid buy/sell power (kW)
$\eta_{charge}, \eta_{discharge}$	Battery efficiency
$s_{initial}$	Initial battery state of charge

Variables

x_t^{buy}	Energy bought from the grid	(Continuous)
x_t^{sell}	Energy sold to the grid	(Continuous)
x_t^{charge}	Energy charged into battery	(Continuous)
$x_t^{discharge}$	Energy discharged from battery	(Continuous)
s_t	Battery state of charge	(Continuous)
y_t^{charge}, $y_t^{discharge} \in \{0, 1\}$	Whether charging or discharging battery	(Binary)

Objective Function

$$\sum_{t=0}^{T}\left(C_t^{sell}x_t^{sell} - C_t^{buy}x_t^{buy}\right) \rightarrow max$$

Constraints

For each hour $t = 0, 1, 2, 3, \dots, T$

- <u>Power balance:</u>

$$E_t^{demand} = E_t^{solar} + x_t^{buy} + x_t^{discharge} - x_t^{charge} - x_t^{sell}$$

- Battery state of charge:

$$s_t = \begin{cases} s_{\text{initial}} + \eta_{\text{charge}} x_t^{charge} - \frac{x_t^{discharge}}{\eta_{\text{discharge}}} & t = 0 \\ s_{t-1} + \eta_{\text{charge}} x_t^{charge} - \frac{x_t^{discharge}}{\eta_{\text{discharge}}} & t \geq 1 \end{cases}$$

$$s_t \leq E^{cap} \quad , s_{initial} \quad given$$

- Battery charge/discharge limits:

$$x_t^{charge} \leq P_{max}^{charge} y_t^{charge}$$

$$x_t^{discharge} \leq P_{discharge}^{max} y_t^{discharge}$$

- No simultaneous battery charging and discharging:

$$y_t^{charge} + y_t^{discharge} \leq 1$$

$$y_t^{charge}, y_t^{discharge} \in \{0,\ 1\}$$

- Grid buy/sell limits:

$$x_t^{buy} \leq P_{buy}^{max}$$

$$x_t^{sell} \leq P_{sell}^{max}$$

- Non-negativity:

$$x_t^{buy},\ x_t^{sell}, x_t^{charge},\ x_t^{discharge},\ s_t,\ y_t^{charge},\ y_t^{discharge} \geq 0$$

2.2.2 Solution Methodology:

For each hour t, the described MILP contains 5 continuous variables and 2 binary variables. Therefore, for a 24 hour period, the formulated MILP contains 120 continuous variables and 48 binary variables.

This MILP formulation is necessary because the system contains both continuous variables (battery SOC and energy bought, sold, charged, or discharged) and binary variables (whether the battery is charging or discharging).

The objective of the model is to maximize profit (or equivalently, minimize the electricity cost) while satisfying all constraints

Solver and Algorithm:

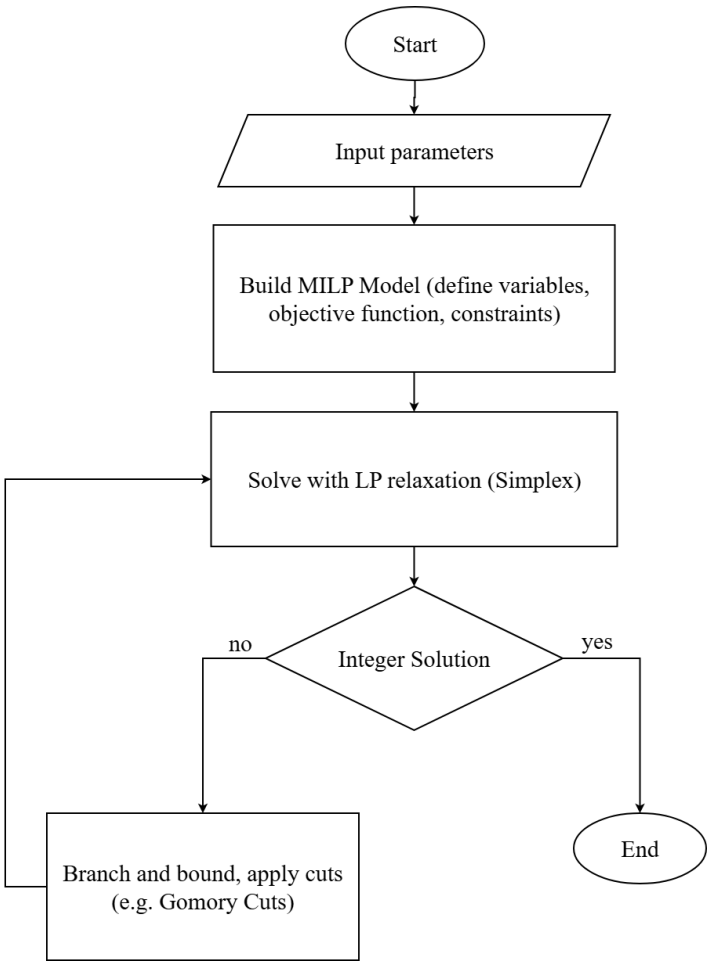

Figure 2.2: Flowchart of the solution algorithm for the linear model

The model is implemented in Python using the PuLP library, which uses the CBC solver. The CBC solver then finds the optimal solution using the following steps:

LP Relaxation: All binary variables are temporarily relaxed to continuous values. The relaxed linear program is solved using the Simplex method.

Branch-and-Bound: If any binary variable in the LP solution is fractional, the solver creates two branches: one where the variable is forced to 0, and another where it is forced to 1. This process continues recursively, removing branches that cannot improve the current best solution.

Gomory Cuts: CBC adds Gomory cuts, which are linear constraints that eliminate fractional solutions. These cuts work by adding a new linear constraint that cuts off the fractional solution but doesn't exclude any feasible integer solutions [12].

2.3. Nonlinear Programming Model:

2.3.1 Mathematical Model:

For the nonlinear case, we keep the same variables and objective function of the linear model and we introduce a new parameter:

k	Loss coefficient

We also adjust the battery SOC constraint to be nonlinear:

- <u>Battery state of charge:</u>

$$E_t^{loss} = k \cdot \left(\frac{(x_t^{charge})^2}{P_{charge}} + \frac{(x_t^{discharge})^2}{P_{discharge}} \right)$$

$$s_t = \begin{cases} s_{\text{initial}} + \eta_{\text{charge}} x_t^{charge} - \frac{x_t^{discharge}}{\eta_{\text{discharge}}} - E_t^{loss} & t = 0 \\ s_{t-1} + \eta_{\text{charge}} x_t^{charge} - \frac{x_t^{discharge}}{\eta_{\text{discharge}}} - E_t^{loss} & t \geq 1 \end{cases}$$

$$s_t \leq E^{cap} \quad , s_{initial} \ given$$

All other constraints remain the same

2.3.2 Solution Methodology:

Adding the nonlinear constraint turns the original MILP into a convex Mixed-Integer Nonlinear Program (MINLP) with 120 continuous variables, 48 binary variables and 24 convex nonlinear constraints for the 24 hour case

Solver and Algorithm:

The problem is solved using the GEKKO library in python. GEKKO solves it to global optimality using the Branch and Bound with SQP as the continuous NLP solver. This combination works best for MINLPs of this size.

The algorithm uses the following steps:

1. Initialization: Relax all binary variables (y_t^{charge}, $y_t^{discharge}$) to continuous variables in their bounds [0,1] creating a relaxed nonlinear programming problem.

2. Node processing in Branch-and-Bound tree: Select an open node from the B&B tree and solve the current node's NLP relaxation using Sequential Quadratic Programming (SQP).

3. Bounding and fathoming

- *If* the NLP is **infeasible**, *then* fathom (prune) the node.
- *If* the relaxed objective value (upper bound) ≤ current best integer feasible lower bound *then* fathom the node (by bound).
- *If* the solution has all binary values for y_t^{charge} and $y_t^{discharge}$ *then* it's a new candidate integer solution, update the best lower bound if better.

4. Branching (if needed)

- If any integer/binary variable is **fractional** *then* select a branching variable.
- Create **two child nodes**:
  - One where $y_t = 0$
  - One where $y_t = 1$
- Add children to the active node list.

Continue until no active nodes remain (tree exhausted). Return the best integer-feasible solution found.

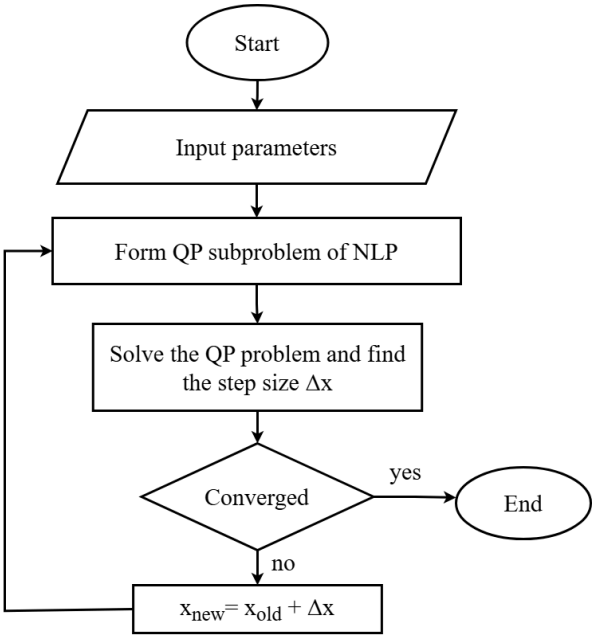

Figure 2.3: Flowchart of the SQP algorithm

Figure 2.4: Flowchart of branch and bound algorithm of the nonlinear model

Chapter 3: Software Implementation and Code Structure

3.1 Code Structure

```
solar-battery-optimization
    README.md
    LICENSE
    .gitignore
    src
        main.py
        time_analysis.py
        constants.py
        profiles.py
        linear.py
        non_linear.py
        charts.py
    results
        solar_vs_demand.png
        comparison.png
        linear
            (Linear)_Scenario_1_*.csv, *.png
            (Linear)_Scenario_2_*.csv, *.png
            (Linear)_sensitivity_analysis.png
        non_linear
            (Non-linear)_Scenario_1_*.csv, *.png
            (Non-linear)_Scenario_2_*.csv, *.png
            (Non-linear)_sensitivity_analysis.png
```

Figure 3.1: File structure of the software implementation

The optimization models are implemented in Python using a modular structure to separate data, model logic, and visualization. The codebase consists of the following modules:

➢ **constants.py**

Purpose: Defines all physical and economic parameters for the 24-hour microgrid model. A single file is used so that both the linear and nonlinear models use the same data.

Contents: Time *T*, grid buy price *C_buy* time-of-use tariff, solar generation *Esolar* and load demand *Edemand*, battery capacity *Ecap*, maximum charge/discharge power *Pcharge_max* and *Pdischarge_max*, round-trip efficiencies *charge_eff* and discharge_eff, initial state of charge *s0*, grid limits *Pbuy_max* and *Psell_max,* loss coefficient *k*.

- **profiles.py**

  Purpose: Builds 24-hour sell-price (export tariff) profiles used as input to the optimization functions. This ensures consistent tariff shapes across different scenarios.

  Function: *variable_tariff_profile(off_peak_price, peak_price)* returns a list of 24 prices. Off-peak is applied to hours 0-6 and 23; peak to hours 7-22.

- **linear.py**

  Purpose: Implements the mixed-integer linear program (MILP) for the 24-hour problem. Solves for optimal grid buy/sell, battery charge/discharge, and state of charge to maximize net revenue.

  Libraries used: *PuLP*, *CBC Coin-or Branch and Cut* (open-source MILP solver, invoked via *PULP_CBC_CMD* with *msg=False*). *pandas* (used to build the results DataFrame). *charts* (module in this project for plotting when *save_results* is *True*).

  Main function: *solve_scenario(C_sell, scenario_name, results_folder, save_results=True)*.

  - Inputs: *C_sell* is the 24-hour sell-price profile; *scenario_name* and *results_folder* for labelling and saving; *save_results* controls whether to write CSV and generate charts.

  - Output: If *save_results* is *True*, writes CSV and calls chart functions and returns a pandas DataFrame. If *save_results* is *False*, returns a dict with exported, *actual_cost*, and *baseline_cost* (used by sensitivity and timing scripts).

- **non_linear.py**

  Purpose: Implements the mixed-integer nonlinear program (MINLP) for the same problem, with an added quadratic loss term in the battery state-of-charge dynamics.

  Libraries used: *GEKKO* (Python package for formulation and solution of dynamic and nonlinear optimization problems), *APOPT* (nonlinear solver; selected with *m.options.SOLVER = 1* and *m.solve(disp=False)*; runs locally with *GEKKO(remote=False))*, *pandas* (results DataFrame), *charts* (same as in *linear.py* when saving results).

Main function: *solve_scenario(C_sell, scenario_name, results_folder, save_results=True*).

Inputs and outputs are the same as *linear.solve_scenario* for fair comparison and reuse in *main.py*, sensitivity, and timing.

➢ **charts.py**

Purpose: Contains all plotting and visualization functions for the project. Uses the same data structures (DataFrames and price arrays) so that linear and nonlinear results can be compared visually.

Libraries used: *Matplotlib* (all figures). NumPy (for array operations in the comparison chart). profiles (*variable_tariff_profile* for the sensitivity analysis).

Functions:

- *solar_vs_demand_chart*: 24-hour bar/line plot of solar generation vs. load demand (input data overview).
- *scenario_chart*: hourly time series of solar, demand, buy, sell, and SOC for one scenario.
- *costs_chart*: pie chart of grid purchase cost vs. export revenue.
- *sources_chart*: stacked bars of solar, grid import, and battery discharge supplying the load, with demand overlay.
- *decision_variables_chart*: binary charge/discharge indicators over time.
- *battery_charging_chart*: charge/discharge bars and SOC trajectory.
- *comparison_chart*: grouped bar chart comparing total solar, buy, discharge, and sell for all four runs (linear/nonlinear times scenario 1/2).
- *sensitivity_chart*: for a given solver, generates sell prices from 22 to 51 AMD/kWh (30 points), calls solve_scenario with *save_results=False* for each, and plots exported energy and actual cost vs. price.

- **main.py**

  Purpose: Top-level script that runs the full workflow: create output directories, plot solar vs. demand, run both scenarios with both solvers, produce the comparison chart, and run the sensitivity analysis for each solver.

  Libraries used: *os*, and project modules *linear*, *non_linear*, *charts*, *constants*, and *profiles* for data and tariff profiles.

  Flow:

  1. Create results/, results/linear/, results/non_linear/.
  2. Call charts.solar_vs_demand_chart with data from constants.
  3. Build scenario 1 and scenario 2 sell profiles via *variable_tariff_profile*.
  4. Run linear solver for both scenarios (with CSV and charts).
  5. Run nonlinear solver for both scenarios (with CSV and charts).
  6. Call *charts.comparison_chart* with the four result DataFrames.
  7. Call *charts.sensitivity_chart* for linear and nonlinear (each runs 30 price points, no per-scenario charts).

- **time_analysis.py**

  Purpose: Dedicated solver timing benchmark. Runs the same 24-hour scenario multiple times with each solver, with no CSV or chart output, and reports average solve time and the ratio. Used to compare computational performance of the MILP vs. MINLP.

  Libraries used: Python standard library *time* (*time.perf_counter()*), *linear*, *non_linear*, and *profiles* (no charts).

  Logic: *N_RUNS = 10* (number of runs per solver).

  1. Build one price profile with *variable_tariff_profile*.
  2. Run *linear.solve_scenario* and *nonlinear.solve_scenario* with *save_results=False* 10 times,
  3. Record each solve time and compute the mean.
  4. Print average time in seconds for each solver and the ratio (nonlinear over linear).

3.2 Inputs and Outputs

All inputs are defined in *constants.py* and can be changed by modifying the values in the file. These inputs represent the system parameters defined for the model and are shared across solvers.

The program generates 3 types of outputs:

- Logs:

When running different scenarios, the numerical results are logged in the console for ease of access.

- CSV files:

For each scenario, a CSV file is exported to the results folder. Exported files consist of 10 columns and 24 rows, each representing the values of parameters or variables at hour *t*.

Column	Hour	Solar	Demand	Buy	Sell	Charge	Discharge	SOC	y_c	y_d
Model	t	E_t^{solar}	E_t^{demand}	x_t^{buy}	x_t^{sell}	x_t^{charge}	$x_t^{discharge}$	s_t	y_t^{charge}	$y_t^{discharge}$

Table 3.1: CSV file columns

CSV (Comma-Separated Values) is a simple plain text data format. It is widely used due to its compatibility with data analysis tools.

- Charts:

Five charts are created for each scenario, one sensitivity chart per model (linear and nonlinear) as well as two general charts, resulting in 22 total all saved in the results folder formatted as PNG.

PNG is the default format used by *matplotlib* when saving figures.

Chapter 4: Case Study: Small Commercial Building in Yerevan

4.1 System parameters

We consider the following system:

- 30 kWh lithium battery, 12 kW charge/discharge power, η = 95 %
- Grid tariffs: buy 38 AMD (00-06 and 23) / 52 AMD (rest) and 2 export price scenarios:
    - Scenario 1: 22 AMD
    - Scenario 2: 35 AMD (00-06 and 23) / 48 AMD (rest)
- Total daily load of 372 kWh
- Total rooftop solar production of 289 kWh
- Max grid buy power in one hour: 60kW, sell power: 30kW
- Initial battery state of charge: 15 kWh

The given parameters are:

E^{cap}	30 kWh	P^{max}_{buy}	60 kW
P^{max}_{charge}, $P^{max}_{discharge}$	12 kW	η_{charge}, η_{dis}	0.95
P^{max}_{sell}	30 kW	s_0	15 kWh

t	E^{solar}_t	E^{demand}_t	P^{buy}_t	P^{sell}_t (Scenario 1)	P^{sell}_t (Scenario 2)
0	0	5	38	22	35
1	0	5	38	22	35
2	0	5	38	22	35
3	0	5	38	22	35
4	0	5	38	22	35
5	0	5	38	22	35
6	0	10	38	22	35
7	1	10	52	22	48
8	4	25	52	22	48
9	10	25	52	22	48

10	18	25	52	22	48
11	28	25	52	22	48
12	36	20	52	22	48
13	40	20	52	22	48
14	41	30	52	22	48
15	38	30	52	22	48
16	32	30	52	22	48
17	22	30	52	22	48
18	12	15	52	22	48
19	5	15	52	22	48
20	2	8	52	22	48
21	0	8	52	22	48
22	0	8	52	22	48
23	0	8	38	22	35

Table 4.1: Case study parameters

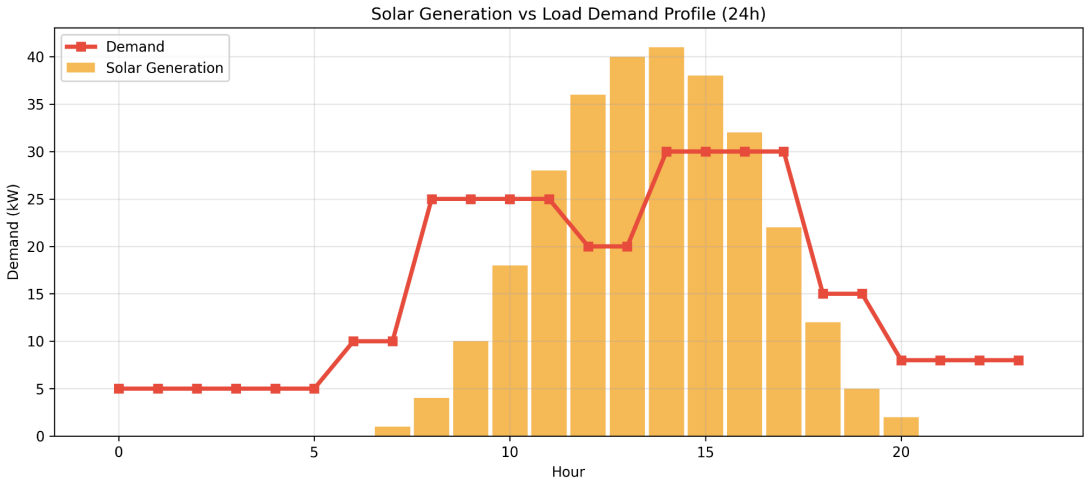

Figure 4.1: Solar generation and load demand profile

For $t = 0, 1, 2, \ldots, 23$, we need to find the optimal solution of the mathematical model described earlier

The objective function value represents the optimized net cost of electricity after the model schedules charging, discharging, buying, and selling. The Amount Saved is computed by comparing this optimized cost to the baseline cost of running the building without solar and battery support.

$$Baseline\ Cost = \sum_{t=0}^{T} E_t^{demand} P_t^{buy} = 18672\ AMD$$

$$Amount\ Saved = Baseline\ Cost - Actual\ Cost$$

4.2 Results

Linear Case - Scenario 1: 22 AMD

t	x_t^{buy}	x_t^{sell}	x_t^{charge}	$x_t^{discharge}$	s_t	y_t^{charge}	$y_t^{discharge}$
0	5	0	0	0	15	0	0
1	5	0	0	0	15	0	0
2	5	0	0	0	15	0	0
3	5	0	0	0	15	0	0
4	5	0	0	0	15	0	0
5	17	0	12	0	26.4	1	0
6	13.79	0	3.79	0	30	1	0
7	0	0	0	9	20.53	0	1
8	20.5	0	0	0.5	20	0	1
9	3	0	0	12	7.37	0	1
10	0	0	0	7	0	0	1
11	0	3	0	0	0	0	0
12	0	16	0	0	0	0	0
13	0	9.42	10.58	0	10.05	1	0
14	0	0	11	0	20.5	1	0
15	0	0	8	0	28.1	1	0
16	0	0	2	0	30	1	0
17	0	0	0	8	21.58	0	1
18	0	0	0	3	18.42	0	1
19	0	0	0	10	7.89	0	1
20	0	0	0	6	1.58	0	1
21	6.5	0	0	1.5	0	0	1
22	8	0	0	0	0	0	0
23	8	0	0	0	0	0	0

Table 4.2: Linear case solution (scenario 1)

Exported Amount: 28.4 kWh

Value of the objective function: -3774.74

Actual Cost: 3774.74 AMD

Amount Saved = Baseline Cost - Actual Cost = 18672 - 3774.74= 14897.26 AMD

Linear Case - Scenario 2: 35 or 48 AMD

t	x_t^{buy}	x_t^{sell}	x_t^{charge}	$x_t^{discharge}$	s_t	y_t^{charge}	$y_t^{discharge}$
0	5	0	0	0	15	0	0
1	5	0	0	0	15	0	0
2	5	0	0	0	15	0	0
3	5	0	0	0	15	0	0
4	5	0	0	0	15	0	0
5	8.79	0	3.79	0	18.6	1	0
6	22	0	12	0	30	1	0
7	0	0	0	9	20.53	0	1
8	20.5	0	0	0.5	20	0	1
9	3	0	0	12	7.37	0	1
10	0	0	0	7	0	0	1
11	0	3	0	0	0	0	0
12	0	16	0	0	0	0	0
13	0	20	0	0	0	0	0
14	0	11	0	0	0	0	0
15	0	8	0	0	0	0	0
16	0	2	0	0	0	0	0
17	8	0	0	0	0	0	0
18	3	0	0	0	0	0	0
19	10	0	0	0	0	0	0
20	6	0	0	0	0	0	0
21	8	0	0	0	0	0	0
22	8	0	0	0	0	0	0
23	8	0	0	0	0	0	0

Table 4.3: Linear case solution (scenario 2)

Exported Amount: 60.0 kWh

Value of the objective function: -3002.00 AMD

Actual Cost: -3002 AMD

Amount Saved = Baseline Cost - Actual Cost = 18672 - 3002 = 15670 AMD

Nonlinear Case - Scenario 1: 22 AMD

We will consider the same cases studied in the linear case for $k = 0.012$

t	x_t^{buy}	x_t^{sell}	x_t^{charge}	$x_t^{discharge}$	s_t	y_t^{charge}	$y_t^{discharge}$
0	5	0	0	0	15	0	0
1	5	0	0	0	15	0	0
2	5	0	0	0	15	0	0
3	5	0	0	0	15	0	0
4	12.96	0	7.96	0	22.5	1	0
5	5	0	0	0	22.5	0	0
6	17.96	0	7.96	0	30	1	0
7	1.9	0	0	7.1	22.47	0	1
8	13.9	0	0	7.1	14.94	0	1
9	7.9	0	0	7.1	7.42	0	1
10	0	0	0	7	0	0	1
11	0	0	3	0	2.84	1	0
12	0	8.8	7.2	0	9.63	1	0
13	0	12.8	7.2	0	16.42	1	0
14	0	3.8	7.2	0	23.21	1	0
15	0	0.8	7.2	0	30	1	0
16	0	2	0	0	30	0	0
17	2.93	0	0	5.07	24.63	0	1
18	0	0	0	3	21.47	0	1
19	4.93	0	0	5.07	16.1	0	1
20	0.93	0	0	5.07	10.73	0	1
21	2.93	0	0	5.07	5.37	0	1
22	2.93	0	0	5.07	0	0	1
23	8	0	0	0	0	0	0

Table 4.4: Nonlinear case solution (scenario 1)

Exported Amount: 28.19 kWh

Value of the objective function: -3801.52 AMD

Actual Cost: 3801.52 AMD

Amount Saved = Baseline Cost - Actual Cost = 18672 - 3801.52 = 14870.48 AMD

Nonlinear Case - Scenario 2: 35 or 48 AMD

t	x_t^{buy}	x_t^{sell}	x_t^{charge}	$x_t^{discharge}$	s_t	y_t^{charge}	$y_t^{discharge}$
0	12.96	0	7.96	0	22.5	1	0
1	12.96	0	7.96	0	30	1	0
2	5	0	0	0	30	0	0
3	5	0	0	0	30	0	0
4	5	0	0	0	30	0	0
5	5	0	0	0	30	0	0
6	10	0	0	0	30	0	0
7	6.16	0	0	2.84	27	0	1
8	18.16	0	0	2.84	24	0	1
9	12.16	0	0	2.84	21	0	1
10	4.16	0	0	2.84	18	0	1
11	0	3	0	0	18	0	0
12	0	16	0	0	18	0	0
13	0	20	0	0	18	1	0
14	0	11	0	0	18	0	0
15	0	8	0	0	18	0	0
16	0	2	0	0	18	0	0
17	5.16	0	0	2.84	15	0	1
18	0.16	0	0	2.84	12	0	1
19	7.16	0	0	2.84	9	0	1
20	3.16	0	0	2.84	6	0	1
21	5.16	0	0	2.84	3	0	1
22	5.16	0	0	2.84	0	0	1
23	8	0	0	0	0	0	0

Table 4.5: Nonlinear case solution (scenario 2)

Exported Amount: 60.0 kWh

Value of the objective function: -3011.06 AMD

Actual Cost: 3011.06 AMD

Amount Saved = Baseline Cost - Actual Cost = 18672 - 3011.06 = 15660.94 AMD

4.3 Discussion

4.3.1 Interpretation of results

We compare the two models (Linear and Nonlinear) on two export price scenarios. The nonlinear model includes a quadratic loss constraint that better represents real battery and inverter behavior. The comparison allows us to evaluate the impact of nonlinear losses on the operational schedule or the saved costs.

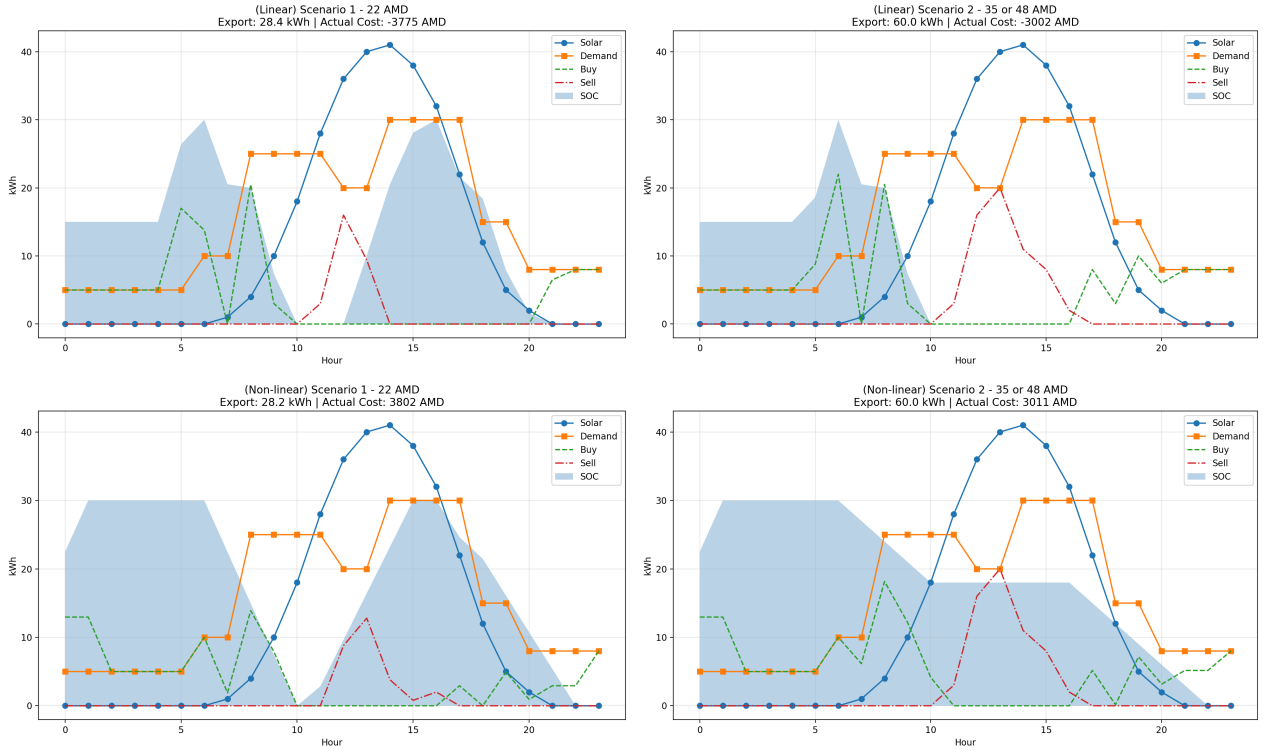

Figure 4.2: Cost charts for linear (top) and nonlinear (bottom) cases for both scenarios

In scenario 2, the model prioritizes selling over charging the battery. The model exclusively sells during peak hours.

Compared to the linear case, in scenario 2, the battery SOC decreases slowly throughout the day. Whereas in other scenarios, the battery completely discharges at hour 10.

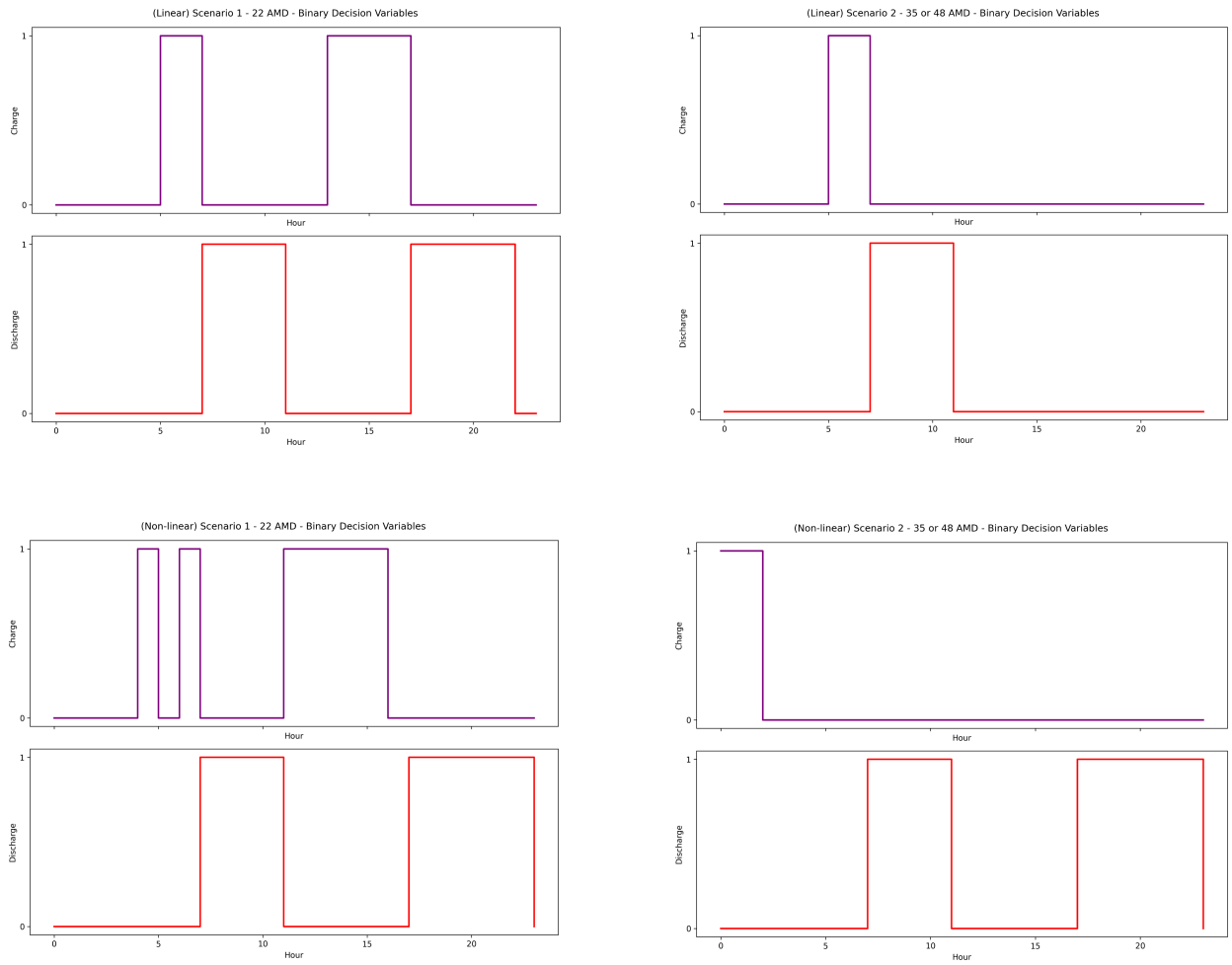

Figure 4.3: Charge/discharge decision variables for linear (top) and nonlinear (bottom) cases for both scenarios

In scenario 1, the model charges the battery during hours when electricity is cheap or when solar generation is high. When solar output drops, the model discharges the battery to reduce grid purchases. This behaviour confirms that the optimization correctly prioritizes low-cost charging and high-value discharging to maximize overall savings. This charging-discharging pattern reflects energy arbitrage in time-of-use electricity markets. By storing energy during low-price periods and discharging during high-price periods, the battery practically shifts energy consumption in time. This behaviour demonstrates that the optimization model successfully takes advantage of the economic incentives in the tariff structure and translates them into an optimal operational schedule.

In scenario 2, the model only charges the battery during non-peak hours. During peak hours, unlike in the first scenario, the model does not charge the battery at all. Instead, it sells almost all excess solar energy directly to the grid because the high selling price makes exporting more profitable than storing the energy. This behavior is confirmed by the binary variables showing that the battery neither charges or discharges starting from hour 11 in the linear case

and during hours 11-16 in the non linear case, meaning all exported energy comes from real-time solar production rather than stored energy in the battery. This highlights an important trade-off between storage and direct export. Instead of storing excess solar generation for later use, the optimization model prefers immediate export because the financial return from selling electricity exceeds the potential benefit of future self-consumption. Meaning, when export prices are sufficiently high, the cost of storing energy increases.

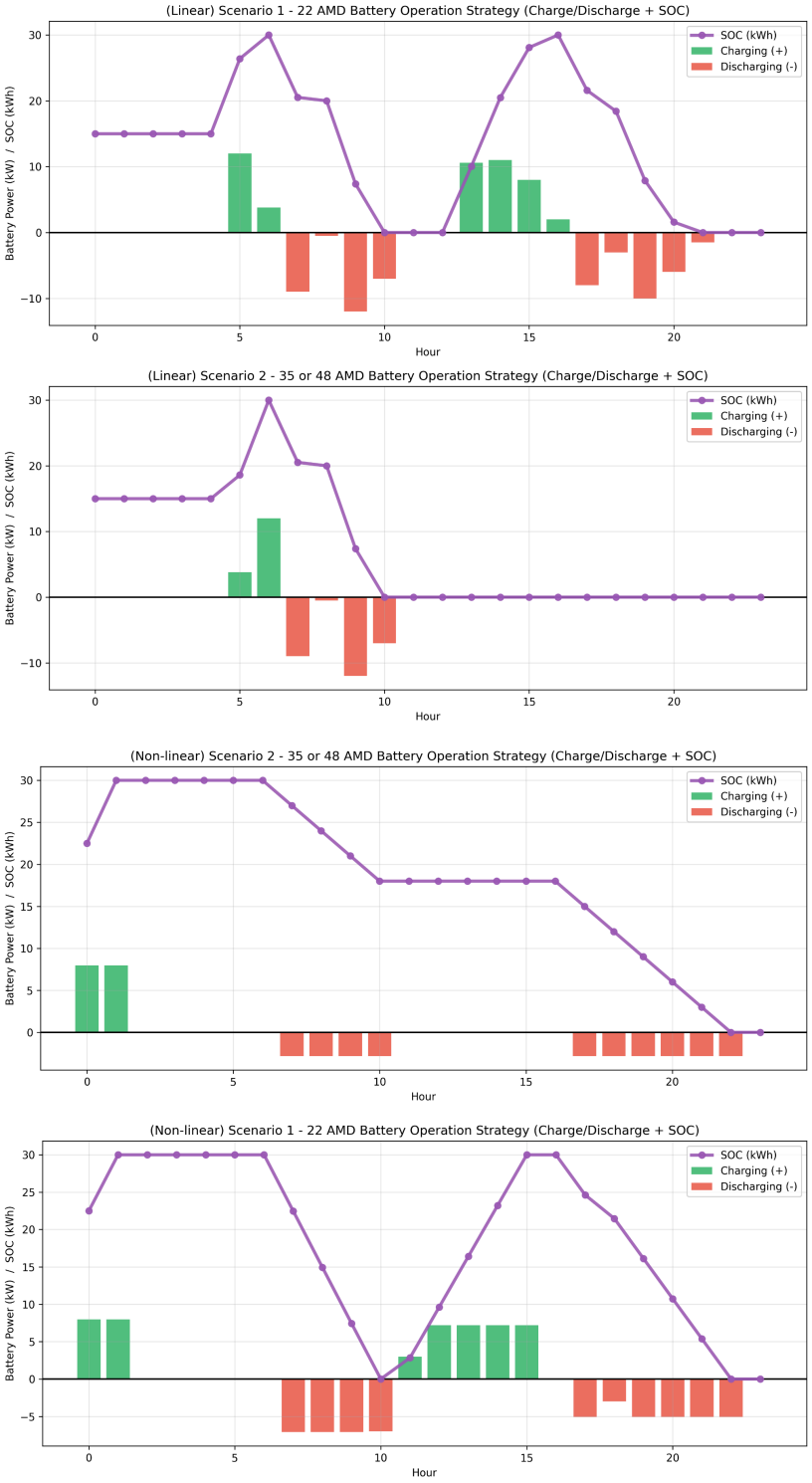

Figure 4.4: Battery operation strategies

The two tariff scenarios show a clear relationship between selling price and the optimal behavior of the microgrid.

When the selling price is 22 AMD/kWh (Scenario 1), the model exports 28 kWh to the grid. When the selling price increases to 35 or 48 AMD/kWh (Scenario 2), the exported energy rises to 60.0 kWh.

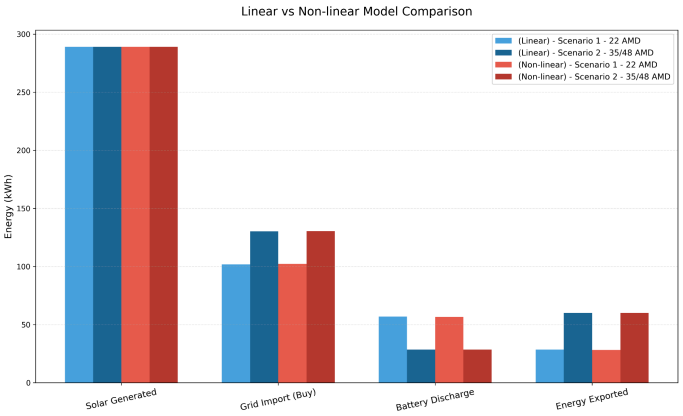

Figure 4.5: Linear vs. nonlinear models

This indicates that higher selling prices cause the model to prioritize exporting energy rather than self-consumption. This behaviour shows how sensitive the optimal scheduling is to electricity pricing. Even relatively small increases in export tariffs can significantly change energy flow patterns within the system. From a policy perspective, this suggests that tariffs and export compensation mechanisms play a crucial role in determining whether energy resources are primarily used for self-consumption or for grid supply. In scenario 2, the optimizer buys more energy from the grid at non-peak hours, charges the battery and then sells at peak hours when it is more profitable.

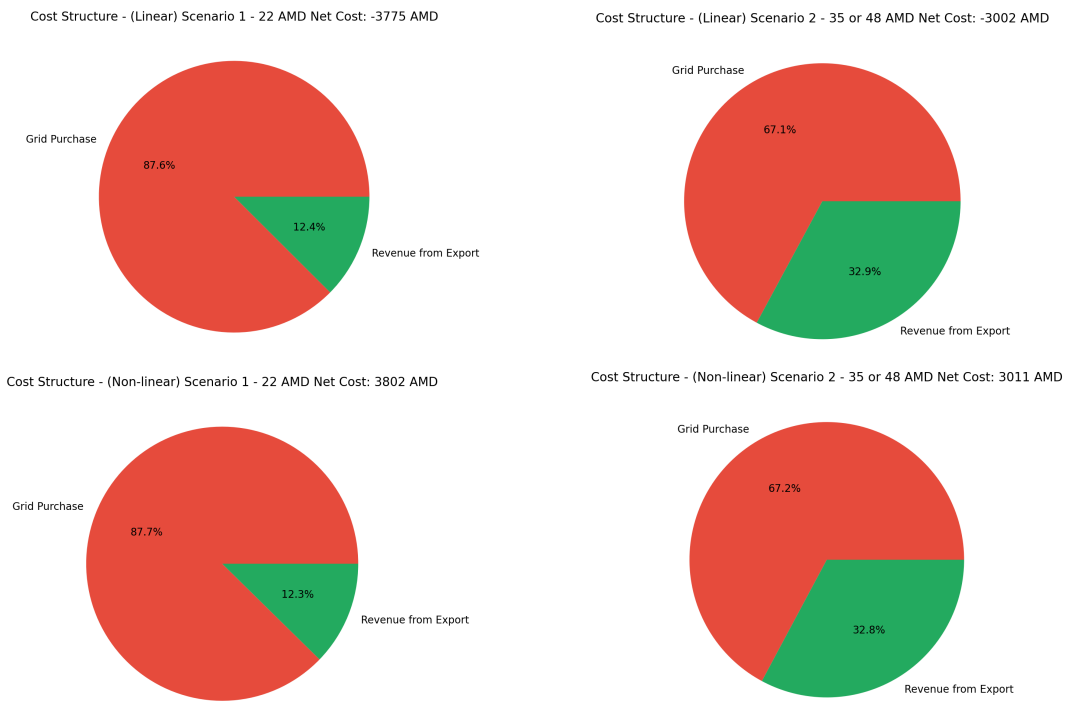

Figure 4.5: Cost structure for linear (top) and nonlinear (bottom) cases for both scenarios

In Scenario 2, both models yield similar operating strategies and therefore the same amount of exported energy (60 kWh). However, the nonlinear model explicitly accounts for quadratic inverter and battery losses. These physical losses represent energy that is consumed internally

and does not reach the building load or the grid. To satisfy the hourly energy balance under the presence of these additional losses, the nonlinear model purchases more electricity from the grid compared to the linear case. This small increase in grid purchases directly demonstrates the real economic penalty of inverter and battery inefficiency that is not visible in linear models. This difference is seen through the approximately 0.1% shift in the cost structure chart between the linear and nonlinear cases in both export-price scenarios. Although the numerical appears small, the inclusion of nonlinear losses improves the physical realism of the model. Real batteries and inverters exhibit efficiency losses that depend on power levels, temperature, and operating conditions. By incorporating these nonlinear characteristics, the MINLP formulation provides a more accurate representation of system behaviour and prevents the model from using unrealistic efficiency assumptions.

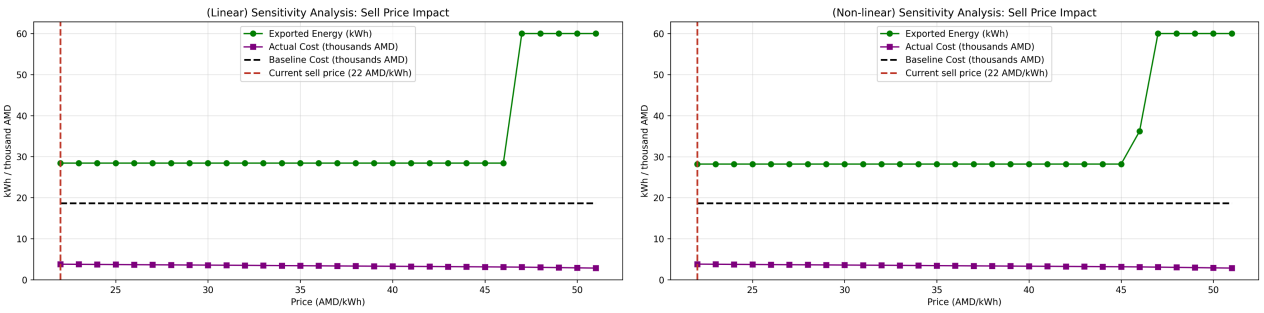

Figure 4.6: Sensitivity analysis for linear (left) and nonlinear (right) cases for both scenarios

When looking at the sensitivity charts, we conclude that increasing the sell price leads to increasingly higher exports. Once the sell price becomes greater than the buy price, exporting becomes more profitable than storing or self-consuming energy. This results in a rise in exported energy as seen in the figure.

Overall, the results demonstrate that the optimal management of photovoltaic generation and battery storage depends on both the tariff prices. The comparison between linear and nonlinear formulations confirms that including realistic loss terms slightly reduces economic performance but produces more reliable results.

4.3.2 Model Assumptions and Limitations

The optimization models presented in this thesis relies on several simplifying assumptions that allow the problem to remain computationally possible while still capturing the key operational characteristics of the PV battery system.

First, photovoltaic generation is assumed to be perfectly known in advance. In practice, solar generation is subject to forecasting uncertainty due to differences in weather. The current model therefore represents an ideal scheduling environment where generation profiles are pre determined.

Second, electricity demand is treated as a fixed and known load profile. Real buildings often have unpredictable consumption patterns due to occupant behavior and environmental conditions.

Another simplifying assumption concerns battery performance. Although nonlinear losses are incorporated in the MINLP model, other physical effects such as temperature dependence, battery degradation, and aging are not explicitly modeled. In real energy storage systems, repeated cycling gradually reduces battery capacity and efficiency over time.

4.3.3 Computational Performance of MILP vs MINLP

The computational complexity of the optimization problem differs significantly between the linear MILP formulation and the nonlinear MINLP formulation. Linear mixed-integer problems can typically be solved efficiently using branch-and-bound algorithms combined with linear programming relaxations. In contrast, nonlinear mixed-integer problems introduce additional challenges because nonlinear constraints make it more complex.

To assess the computational trade-off between the linear and nonlinear formulations of our problem, a timing benchmark was conducted. The goal was to compare the raw solve time of each approach under identical conditions, isolating the solver performance from any outputs.

Both formulations were evaluated on the same 24-hour dispatch scenario: a flat export tariff of 22 AMD/kWh with the standard solar, demand, and grid price profiles defined previously. To obtain stable results, each solver was run 10 consecutive times, and the mean execution time was used. The benchmark measured only the optimization solve itself, no CSV export, chart generation, or console output to ensure a fair comparison of solver performance.

Model	Time (avg of 10)
Linear (PuLP/CBC)	0.103s
Nonlinear (GEKKO)	0.485s

Table 4.6: Linear vs. nonlinear model timing

The benchmark was executed on a standard desktop environment. The linear solver (PuLP/CBC) was able to solve a single scenario in approximately 0.1 seconds on average, while the nonlinear solver (GEKKO/APOPT) required approximately 0.5 seconds. The nonlinear formulation was thus around 5 times slower than the linear one for this problem size.

The MILP model solved relatively quickly due to the linear structure of the objective function and constraints. The solver was able to identify optimal charging and discharging schedules within a short computation time.

The MINLP model required greater computational effort due to the presence of quadratic loss terms. Nonlinear constraints increase the difficulty of the optimization problem because the solver must evaluate nonlinear relationships while also handling integer decision variables.

This trade-off between computational speed and modeling accuracy is a common consideration in optimization problems. Linear models are often preferred for large-scale planning problems due to their simplicity, while nonlinear formulations are more valuable when realistic representation of systems is needed.

4.3.4 Practical Implications

These results provide several insights for the design and operation of photovoltaic battery systems in grid-connected buildings.

First, the optimization results demonstrate that battery storage can effectively reduce electricity costs by shifting energy consumption from high-price periods to low-price periods.

Second, the results highlight the importance of electricity export pricing. When export prices increase, the optimization model prioritizes exporting excess solar energy instead of storing it for later use.

Third, the comparison between linear and nonlinear models shows that ignoring nonlinear efficiency losses may slightly overestimate the economic benefits of battery storage.

These findings are relevant for energy system planners, building managers, and policymakers who are evaluating the economic value of renewable energy systems. By optimizing energy management, buildings can improve the utilization of locally generated renewable energy and reduce reliance on grid electricity during peak demand hours.

Conclusion

This thesis developed a mathematical model to find the optimal daily energy management schedule of a grid-connected photovoltaic battery system under time-of-use electricity tariffs. The objective was to minimize the total electricity cost of a commercial building by determining the best hourly decision for purchasing electricity from the grid, exporting surplus solar energy, and charging or discharging a lithium-ion battery.

Two mathematical optimization models were developed and compared. The first model formulated the problem as a mixed-integer linear program (MILP) assuming constant battery efficiency. The second model extended the formulation to a convex mixed-integer nonlinear program (MINLP) by introducing a quadratic loss term that represents realistic inverter and battery losses. Both models were implemented in Python using open-source optimization tools: PuLP with the CBC solver for the linear case, and GEKKO with the APOPT solver for the nonlinear case.

A case study based on a commercial building in Yerevan was used to evaluate the models under two different export compensation scenarios. The results show that the optimized system significantly reduces electricity costs compared to a baseline grid-only operation. The optimal schedules show expected energy management behavior: the battery charges during periods of low electricity prices or excess solar generation and discharges during high-price periods to reduce grid purchases.

The comparison between the linear and nonlinear models highlights the importance of accounting for realistic system losses. While the linear model slightly overestimates savings due to the assumption of constant efficiency, the nonlinear formulation provides a more accurate representation of real system behavior. Nevertheless, the linear model remains useful as a quick efficient approximation.

The developed optimization model helps us realize how mathematical programming can be used to support energy management decisions for energy systems. The results suggest that combining photovoltaic generation with battery storage can substantially reduce costs of a grid connected building operating under time-of-use tariffs.

References

1. APOPT Solver. (n.d.). Retrieved November 26, 2025, from https://github.com/APMonitor/apopt

2. Bean, R., & Khan, H. (2018). Using solar and load predictions in battery scheduling at the residential level. *Conference Paper / Journal Name* [please insert journal or conference], 2018.

3. CBC (COIN-OR Branch and Cut) Solver. (n.d.). Retrieved November 26, 2025, from https://github.com/coin-or/Cbc

4. COIN-OR. (n.d.). *PuLP: A linear programming toolkit for Python*. Retrieved March 1, 2026, from https://coin-or.github.io/pulp/main/includeme.html

5. Cornell University. (2021). *Branch and bound (BB) for MINLP*. Retrieved March 1, 2026, from https://optimization.cbe.cornell.edu/index.php?title=Branch_and_bound_(BB)_for_MINLP

6. Cornell University. (2021). *Simplex algorithm*. Retrieved March 1, 2026, from https://optimization.cbe.cornell.edu/index.php?title=Simplex_algorithm

7. Electric Networks of Armenia. (n.d.). Retrieved November 22, 2025, from https://www.ena.am/Info.aspx?id=11

8. GeeksforGeeks. (n.d.). *Introduction to branch and bound*. Retrieved March 1, 2026, from https://www.geeksforgeeks.org/dsa/introduction-to-branch-and-bound-data-structures-and-algorithms-tutorial/

9. GeeksforGeeks. (n.d.). *Introduction to Pandas in Python*. Retrieved March 1, 2026, from https://www.geeksforgeeks.org/pandas/introduction-to-pandas-in-python/

10. Gekko Development Team. (n.d.). *Gekko overview*. Retrieved March 1, 2026, from https://gekko.readthedocs.io/en/latest/overview.html

11. Hexaly. (n.d.). *Branch and cut algorithms*. Retrieved March 1, 2026, from https://www.hexaly.com/algorithms/branch-and-cut

12. Hovhannisyan, I. (2025). *Mathematical programming* [Lecture notes]. National Polytechnic University of Armenia.

13. Hunter, J., et al. (n.d.). *Matplotlib tutorials*. Matplotlib.org. Retrieved March 1, 2026, from https://matplotlib.org/stable/tutorials/index.html

14. Kleissl, J., Nottrott, A., & Washom, B. (2013). Energy dispatch schedule optimization and cost-benefit analysis for grid-connected photovoltaic-battery storage systems. *Renewable Energy, 55*, 230-240. https://doi.org/10.1016/j.renene.2012.12.036

15. Parkinson, A. R., Balling, R., & Hedengren, J. D. (2018). *Optimization methods for engineering design* (2nd ed.). Brigham Young University.

16. Python Software Foundation. (n.d.). Retrieved March 1, 2026, from https://www.python.org/

17. W3Schools. (n.d.). Retrieved March 1, 2026, from https://www.w3schools.com/